

```

# =====
# Name: Srushti Kale
# PRN: 23070521067
# Practical 8: Stock Market Prediction using RNN/LSTM
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout

# 1. Generate Synthetic NASDAQ Stock Price Data (or load from CSV/yfinance)
np.random.seed(42)
days = 1000
time_series = np.sin(np.linspace(0, 50, days)) * 20 + np.linspace(100, 250, days) + np.random.normal(0, 2, days)
df = pd.DataFrame(time_series, columns=['Close'])

# 2. Data Preprocessing & Normalization
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df[['Close']].values)

# Create sequences for training (sliding window approach)
def create_sequences(data, sequence_length=60):
    X, y = [], []
    for i in range(sequence_length, len(data)):
        X.append(data[i-sequence_length:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)

SEQ_LEN = 60
X, y = create_sequences(scaled_data, sequence_length=SEQ_LEN)

# Train-Test Split (80% Train, 20% Test)
train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

# Reshape input to [samples, time steps, features] for LSTM
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

# 3. Build LSTM Network
model = Sequential([
    LSTM(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)),
    Dropout(0.2),
    LSTM(units=50, return_sequences=False),
    Dropout(0.2),
    Dense(units=25),
    Dense(units=1)
])

# 4. Compile and Train Model
model.compile(optimizer='adam', loss='mean_squared_error')
model.summary()

history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=32,
    validation_data=(X_test, y_test),
    verbose=1
)

# 5. Predictions and Inversion of Scaling
predictions = model.predict(X_test)
predictions = scaler.inverse_transform(predictions)
actual_prices = scaler.inverse_transform(y_test.reshape(-1, 1))

# 6. Visualize Predicted vs Actual Values
plt.figure(figsize=(12, 6))
plt.plot(actual_prices, color='blue', label='Actual Stock Price')
plt.plot(predictions, color='red', label='Predicted Stock Price (LSTM)')
plt.title('Stock Market Prediction using LSTM')
plt.xlabel('Time (Days)')
plt.ylabel('Price')
plt.legend()
plt.grid(True)
plt.show()

```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to the `Sequential` model constructor. Please use `Model.compile()` instead.
super().__init__(**kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 50)	10,400
dropout (Dropout)	(None, 60, 50)	0
lstm_1 (LSTM)	(None, 50)	20,200
dropout_1 (Dropout)	(None, 50)	0
dense (Dense)	(None, 25)	1,275
dense_1 (Dense)	(None, 1)	26

Total params: 31,901 (124.61 KB)

Trainable params: 31,901 (124.61 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/20
24/24 5s 66ms/step - loss: 0.0436 - val_loss: 0.0180
Epoch 2/20
24/24 1s 52ms/step - loss: 0.0059 - val_loss: 0.0025
Epoch 3/20
24/24 1s 52ms/step - loss: 0.0038 - val_loss: 9.2209e-04
Epoch 4/20
24/24 1s 60ms/step - loss: 0.0031 - val_loss: 9.6259e-04
Epoch 5/20
24/24 2s 79ms/step - loss: 0.0033 - val_loss: 0.0052
Epoch 6/20